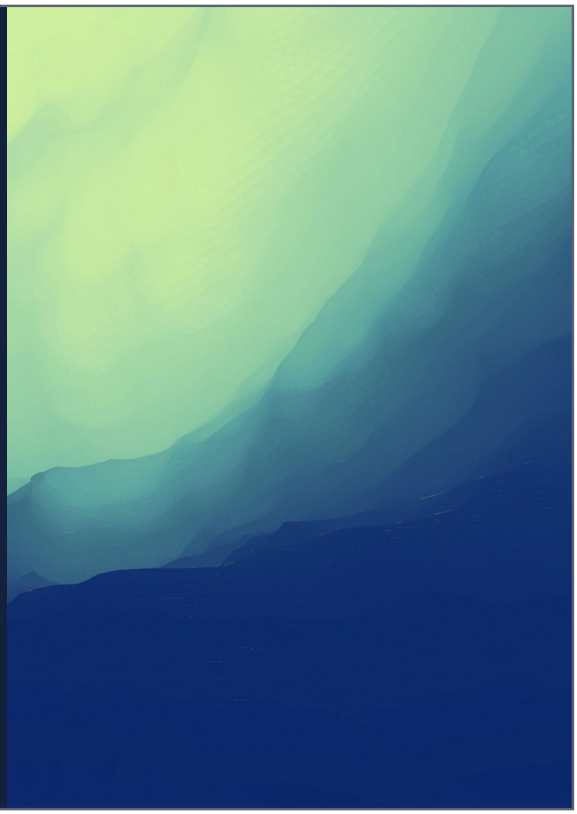
MODULE 03

Unit Testing in Python with pytest

Software quality, test design, unittest, pytest fixtures, mocking, coverage and TDD

```
>>> assert sorted([3, 1, 2]) == [1, 2, 3]
```



AIN3005 · Advanced Python Programming

Binnur Kurt, PhD · Bahçeşehir University · AI Engineering

MODULE 03 · UNIT TESTING

Learning Objectives

By the end of this module, you will be able to:

1. Explain software quality with ISO/IEC 25010 and distinguish verification from validation
2. Trace a defect from error to fault to failure and pick QA activities that catch it
3. Classify tests by level, method and type, and choose suitable test doubles
4. Apply principles of good unit tests: AAA, one behaviour per test, F.I.R.S.T.
5. Write and run test suites with `unittest` and with `pytest`
6. Use fixtures, parametrization and markers to keep pytest suites concise
7. Test exceptions and isolate dependencies with `pytest.raises` and `unittest.mock`
8. Measure coverage with pytest-cov and develop code test-first with TDD

Module Roadmap

01 **Software Quality and Testing**
What quality means and how we assure it

02 **Testing Levels, Types and Test Doubles**
Classifying tests and isolating the unit under test

03 **Principles of Good Unit Tests**
Design for testability and write tests that stay useful

04 **unittest Basics**
The xUnit framework in the standard library

05 **pytest Essentials**
Plain functions, plain assert, powerful tooling

06 **Fixtures and Parametrization**
Reusable setup, many cases, fine-grained control

07 **Testing Exceptions and Mocking**
Expected failures and isolated dependencies

08 **Coverage and TDD**
Measure what the tests exercise, let tests drive the design

01

Software Quality and Testing

What quality means and how we assure it

- Consumer and producer views of quality
- ISO/IEC 25010 product quality model
- Verification vs validation
- Defects, static analysis and testing

What Is Software Quality?

Software = programs + related data + documentation, built for a customer or a market

Consumers

- **Customers** acquire the product
- **Users** work with it daily
- Expect *useful functions*, as specified

Producers

- Develop, maintain, market and service it
- Must honour contracts and the **SLA**
- Want a predictable, affordable process

Quality according to Pressman

An effective software process applied in a manner that creates a useful product that provides measurable value for those who produce it and those who use it.

Quality Engineering and Quality Assurance

1 → 2 → 3

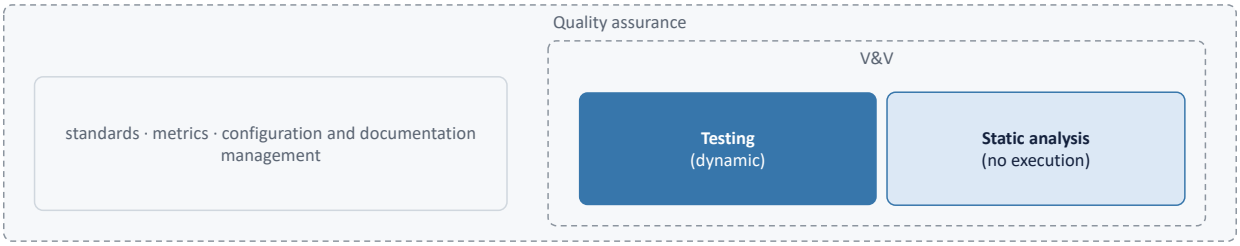
Quality planning
Set quality goals and strategy within cost and schedule

Quality assurance
V&V, standards, metrics, reviews, configuration management

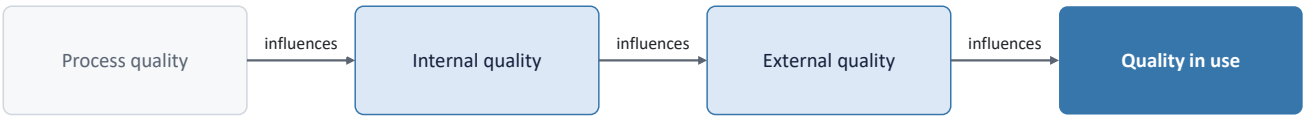
Post-QA
Measure, analyse, give feedback, follow up

Key idea

Quality assurance (QA): a systematic, planned set of actions that gives *adequate confidence* that development and maintenance conform to the specification, the schedule and the budget (Galin, 2004).



ISO/IEC 25010: The Product Quality Model



Functional suitability
complete, correct, appropriate

Performance efficiency
time, resources, capacity

Compatibility
co-existence, interoperability

Interaction capability
renamed (was Usability)

Reliability
faultless, available, recoverable

Security
confidentiality, integrity, authenticity

Maintainability
modular, analysable, **testable**

Flexibility
renamed (was Portability)

Safety
new in 2023

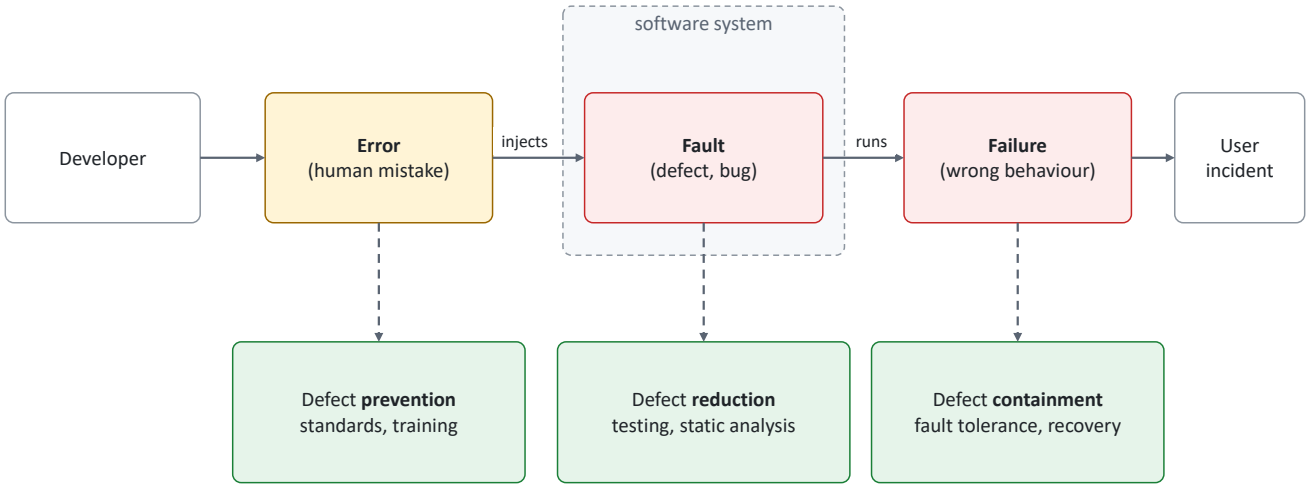
Verification vs Validation

	Verification	Validation
Question	Are we building the product **right**?	Are we building the **right product**?
Checks against	the specification (functional and non-functional requirements)	real stakeholder needs and expectations
Typical activities	reviews, static analysis, unit and integration tests	acceptance tests, prototypes, demos, beta tests
Mainly done by	developers, testers, QA team	customers and end users

Requirements

Functional requirement: what the system must *do* ("transfer money between accounts"). **Non-functional requirement**: how well it does it ("respond within one second") — also called a *quality attribute*.

Error, Fault, Failure: The Defect Chain



Example: misread limit rule → code uses `amount >= limit` → a withdrawal of exactly the limit is refused

Static Analysis: Checking Without Running

Why analyse without running?

- One error cannot hide another, unlike during test execution
- Incomplete programs can be analysed — no test harness needed
- Also checks standards, portability and maintainability

Inspection
Formal examination of artefacts by trained inspectors (Fagan, 1976)

Review
Peers read the change before it is merged: pull-request review

Automated analysis
Linters and type checkers flag suspicious patterns

Key idea
Linters flag **data faults** (unused or undefined names), **control faults** (unreachable code) and **interface faults** (wrong argument types or counts).

Linters and Type Checkers: ruff and mypy

report.py

```python
import os

from banking import Account


def total(accounts: list[Account]) -> float:
    result = 0.0
    for account in accounts:
        result += account.balance
    return reslt


def describe(account: Account) -> str:
    return "Balance: " + account.balance
```

TERMINAL

```
$ ruff check --output-format=concise report.py
report.py:1:8: F401 [*] `os` imported but unused
report.py:10:12: F821 Undefined name `reslt`
Found 2 errors.
[*] 1 fixable with the `--fix` option.
$ mypy --hide-error-codes report.py
report.py:10: error: Name "reslt" is not defined
report.py:14: error: Unsupported operand types for +
    ("str" and "float")
Found 2 errors in 1 file (checked 1 source file)
```

Tip

Run both in CI next to `pytest`: they catch whole bug classes before any test runs.

Tool	Finds	Install
`ruff`	unused imports, undefined names, style	`pip install ruff`
`mypy`	type errors, using your type hints	`pip install mypy`

What Is Software Testing?

Software testing: the *dynamic* evaluation of a program on a *finite* set of *selected* test cases, against the *expected* behaviour

Dynamic

The system under test (**SUT**) is executed with concrete inputs

Finite

Exhaustive testing is impossible — the input space is effectively infinite

Selected

Choose the tests most likely to expose failures

Expected

Each run ends with a verdict: pass or fail

Key idea

Testing can show the presence of bugs, never their absence (after Dijkstra, 1970).

QUIZ

Right Product or Product Right?

The bank's app computes interest exactly as the specification says (monthly), but customers needed daily compounding. Which activity reveals this problem?

- A Verification
- B Validation
- C Static analysis with `mypy`
- D Unit testing the interest function

ANSWER

Right Product or Product Right?

The bank's app computes interest exactly as the specification says (monthly), but customers needed daily compounding. Which activity reveals this problem?

- A Verification
- **B Validation** ✓
- C Static analysis with `mypy`
- D Unit testing the interest function

Why

The code matches its specification, so verification (including unit tests and static analysis) passes. Only validation — checking against real stakeholder needs — reveals that the specification itself is wrong.

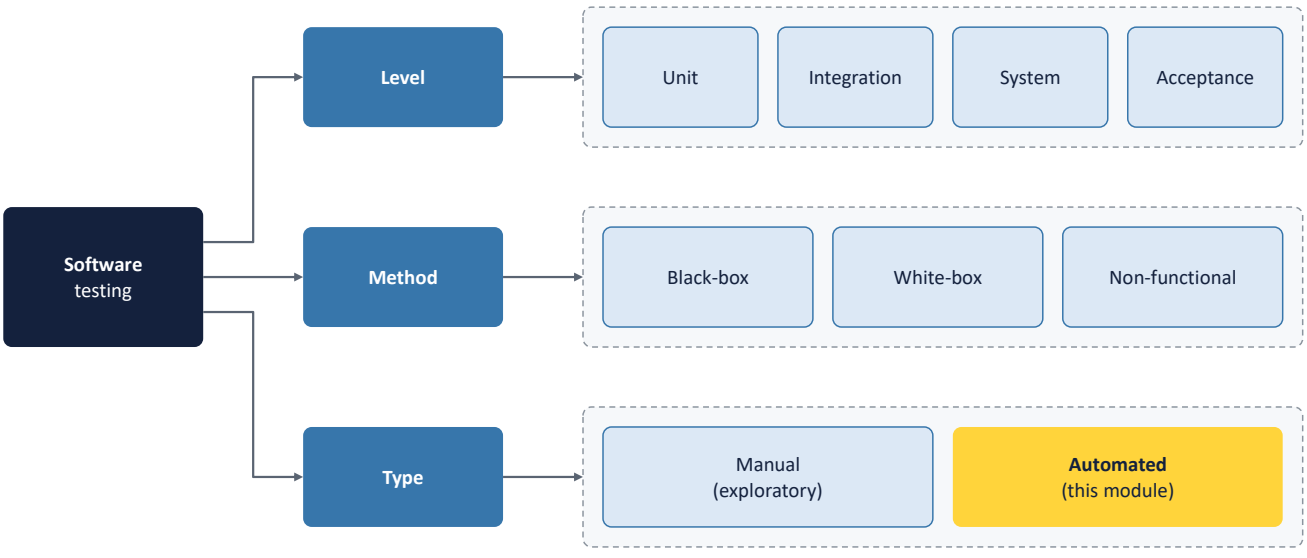
02

Testing Levels, Types and Test Doubles

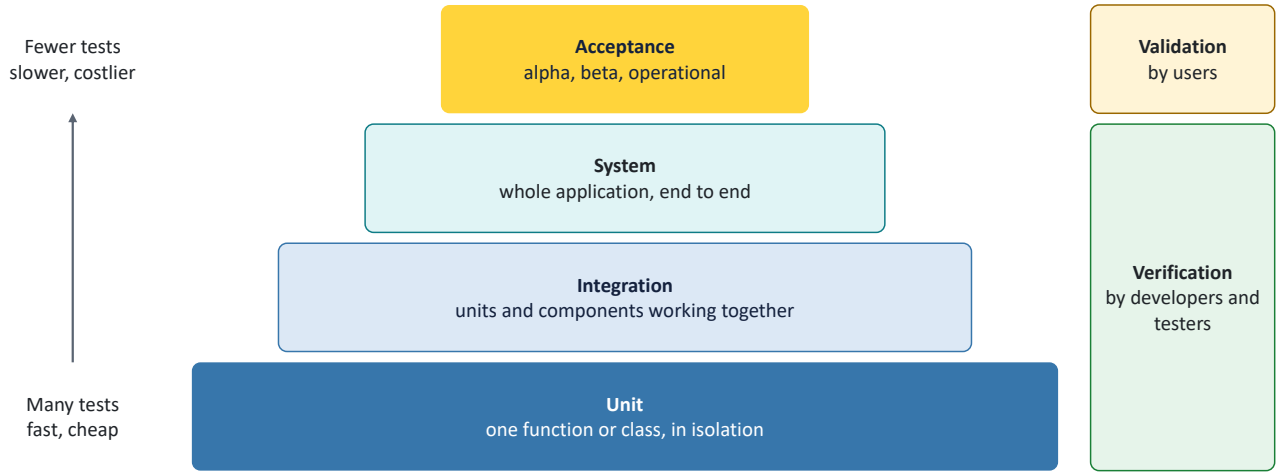
Classifying tests and isolating the unit under test

- Levels and the test pyramid
- Black-box, white-box and non-functional methods
- Partitions, boundaries and coverage
- Dummies, stubs, spies, mocks and fakes

A Taxonomy of Software Testing



Testing Levels and the Test Pyramid



Testing Methods Compared

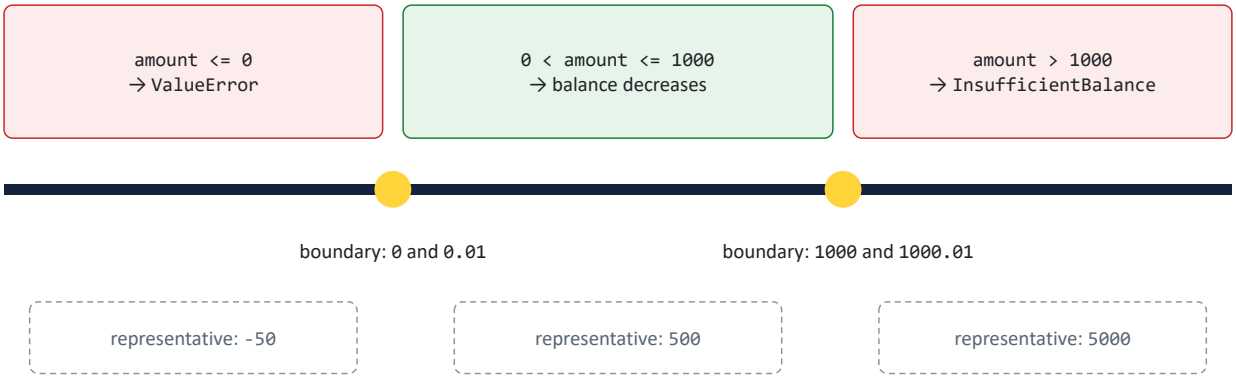
Method	Test cases derived from	Needs the code?	Typical techniques
Black-box (responsibility-based)	specification, requirements	no	equivalence partitioning, boundary values, decision tables, random/fuzz, model-based, smoke, sanity
White-box (implementation-based)	source-code structure	yes	statement, branch, condition and path coverage; fault injection; mutation testing
Grey-box (hybrid)	both	partly	API and integration tests designed with knowledge of internals
Non-functional	quality requirements	no	performance, load, volume, stress, security, usability, accessibility

Key idea

Use both views: black-box finds *missing* behaviour, white-box finds *untested* code.

Equivalence Partitions and Boundary Values

`withdraw(amount)` on an account whose balance is 1000:



Tip

Test one value per partition plus both sides of every boundary: faults cluster at `<` vs `<=`.

White-Box Testing and Coverage Criteria

fee.py

```python
def fee(amount: float, premium: bool) -> float:
    rate = 0.01
    if premium:
        rate = 0.0
    return amount * rate
```

Testing the tests

Mutation testing (`mutmut`) changes the code, e.g. `0.01` → `0.02`, and checks that some test fails. **Fault injection** does the same to the environment.

Criterion	Requires	Tests for `fee`
Statement	every line runs	1: `premium=True`
Branch (decision)	every `if` both ways	2: `True` and `False`
Condition	each boolean sub-expression both ways	2 (one condition)
Path	every route through the code	2
Function	every function called	1

Test Doubles: Replacing Dependencies

A **test double** replaces a *depended-on component* (DOC) so the SUT can be tested in isolation

Double	What it does	Python example
Dummy	fills a parameter list; never actually used	`None`, `object()`
Stub	returns canned answers to calls	`Mock(return_value=35.5)`
Spy	a stub that also records how it was called	`mocker.spy(obj, "method")`
Mock	pre-programmed with expectations; verifies the calls	`m.assert_called_once_with(...)`
Fake	working but simplified implementation	in-memory repository, SQLite `:memory:`

✓ **Tip**

`unittest.mock.Mock` can play every role except the fake.

Test Doubles Without a Library

doubles.py

```python
from types import SimpleNamespace


def alert_if_low(account, notifier, limit=100):
    if account.balance < limit:
        notifier.send(account.iban, "Low balance")


class NotifierSpy:
    def __init__(self):
        self.sent = []

    def send(self, to, text):
        self.sent.append((to, text))


spy = NotifierSpy()
alert_if_low(SimpleNamespace(iban="TR1", balance=40), spy)
alert_if_low(SimpleNamespace(iban="TR2", balance=900), spy)
print(spy.sent)
```

OUTPUT

```
[('TR1', 'Low balance')]
```

ⓘ The `SimpleNamespace` accounts are **stubs**: canned `iban` and `balance`.

★ `NotifierSpy` is a **spy**: it records calls instead of sending messages.

✓ No library needed — duck typing: any object with `send` will do.

QUIZ

Name That Test Double

Tests replace the PostgreSQL account repository with a class that keeps accounts in a `dict` and really saves and finds them. Which kind of test double is it?

- A Dummy
- B Stub
- C Fake
- D Mock

ANSWER

Name That Test Double

Tests replace the PostgreSQL account repository with a class that keeps accounts in a `dict` and really saves and finds them. Which kind of test double is it?

- A Dummy
- B Stub
- C Fake
- D Mock

Why

A fake has a working but simplified implementation. A stub would only return canned answers; a mock would verify the calls made to it.

TRY IT

Choose Test Inputs for `is_valid_age`

`is_valid_age(age)` accepts integer ages from 18 to 65 inclusive.

1. List the equivalence partitions
2. Choose one representative per partition
3. Add the boundary values
4. Which single test would catch `18 < age` instead of `18 <= age`?

ages.py

```
def is_valid_age(age: int) -> bool:
    return 18 <= age <= 65
```

Tip

A boundary has two sides: test the value just inside *and* just outside.

03

Principles of Good Unit Tests

Design for testability and write tests that stay useful

- Principles of test automation
- Arrange–Act–Assert and the four-phase test
- One behaviour per test
- F.I.R.S.T. properties

Principles of Test Automation

Minimize untestable code
Separate logic from GUI, threads, clocks

No test logic in production
No `if testing:` branches

Verify one condition per test
A failure points to one cause

Test concerns separately
One behaviour, one test

Keep tests independent
Any order, any subset

Use the front door first
Test via the public API

Communicate intent
Tests document usage

Easy to set up
Small fixtures, few doubles

Fast
Run them at every change

Key idea
Tests are code too: they deserve the same design care as production code.

Minimize Untestable Code

Hard to test: hidden clock dependency

```python
from datetime import date


def transfer_fee(amount: float) -> float:
    if date.today().weekday() >= 5:
        return round(amount * 0.01, 2)
    return 0.0
```

Testable: the dependency is a parameter

```python
from datetime import date


def transfer_fee(amount: float, day: date) -> float:
    if day.weekday() >= 5:  # Saturday, Sunday
        return round(amount * 0.01, 2)
    return 0.0


print(transfer_fee(500, date(2026, 9, 26)))
print(transfer_fee(500, date(2026, 9, 28)))
```

OUTPUT

```
5.0
0.0
```

No Test Logic in Production Code

payments.py

```python
import os

TESTING = os.environ.get("TESTING") == "1"


def charge(card, amount, gateway):
    if TESTING:  # test hook: never do this
        return "approved"
    return gateway.charge(card, amount)
```

Watch out

Tests must verify the *real* behaviour. Pass a fake `gateway` from the test instead.



Dieselgate (2015): engine software that behaved differently under test

One Behaviour per Test

Avoid: several concerns in one test

```python
def test_account():
    account = Account("TR1", 1000)
    account.deposit(500)
    assert account.balance == 1500
    account.withdraw(200)
    assert account.balance == 1300
    assert account.iban == "TR1"
```

Prefer: one behaviour, a telling name

```python
def test_deposit_increases_balance():
    account = Account("TR1", 1000)
    account.deposit(500)
    assert account.balance == 1500


def test_withdraw_decreases_balance():
    account = Account("TR1", 1000)
    account.withdraw(200)
    assert account.balance == 800
```

Key idea

When a test stops at its first failed assertion, everything after it is unchecked — defect localisation suffers.

Four-Phase Test and Arrange–Act–Assert



Setup · Arrange
Build the fixture: objects and doubles the test needs

Exercise · Act
Call the behaviour under test, once

Verify · Assert
Compare the outcome with the expectation

Teardown
Restore the initial state (often automatic)

test_account.py

```python
def test_withdraw_decreases_balance():
    # Arrange
    account = Account("TR1", 1000)
    # Act
    account.withdraw(300)
    # Assert
    assert account.balance == 700
```


F.I.R.S.T. Properties of Unit Tests

	Property	Meaning	Typical violation
F	Fast	milliseconds per test, run at every change	real network or database calls
I	Independent	no shared state; any order, any subset	module-level objects mutated by tests
R	Repeatable	same result on every machine, every run	`random`, clock, time zones
S	Self-validating	pass or fail automatically, no reading	`print` instead of `assert`
T	Timely	written with (or before) the code	tests added months later

Watch out
A suite nobody trusts is worse than none: slow or flaky tests end up being ignored.

DISCUSS

Which Principles Does This Test Break?

Find at least four problems and suggest a fix for each.

```python
import random

from banking import Account

shared = Account("TR1", 1000)


def test_bank():
    amount = random.randint(1, 2000)
    shared.withdraw(amount)
    print("new balance:", shared.balance)
    shared.deposit(amount)
    assert shared.balance == 1000
```


04

unittest Basics

The xUnit framework in the standard library

- The code under test: a small banking module
- TestCase classes, setUp and assert methods
- Running and discovering tests
- Running tests from PyCharm

The Code Under Test: `banking.py`

banking.py (1/2)

```python
from dataclasses import dataclass
from enum import Enum


class AccountStatus(Enum):
    ACTIVE = "active"
    CLOSED = "closed"


class InsufficientBalance(Exception):
    """Withdrawal larger than the balance."""

    def __init__(self, deficit: float) -> None:
        message = f"balance too low by {deficit}"
        super().__init__(message)
        self.deficit = deficit
```

banking.py (2/2)

```python
@dataclass
class Account:
    iban: str
    balance: float = 0.0
    status: AccountStatus = AccountStatus.ACTIVE

    def deposit(self, amount: float) -> None:
        self._check(amount)
        self.balance += amount

    def withdraw(self, amount: float) -> None:
        self._check(amount)
        if amount > self.balance:
            raise InsufficientBalance(amount - self.balance)
        self.balance -= amount

    def _check(self, amount: float) -> None:
        if self.status is AccountStatus.CLOSED:
            raise ValueError("account is closed")
        if amount <= 0:
            raise ValueError("amount must be positive")
```


unittest at a Glance

- In the standard library since Python 2.1: an xUnit framework modelled on JUnit
- Tests are methods named `test_*` in a `unittest.TestCase` subclass
- Checks use `self.assert*` methods, not the `assert` statement
- `setUp` / `tearDown` run around **every** test method
- Built-in runner: `unittest.main()` or `python -m unittest`

test_sum.py

```python
import unittest


class TestSum(unittest.TestCase):
    def test_sum_of_list(self):
        self.assertEqual(sum([1, 2, 3]), 6)

    def test_sum_of_tuple(self):
        self.assertEqual(sum((1, 2, 2)), 5)


if __name__ == "__main__":
    unittest.main()
```

PYTHON TEST_SUM.PY

```
..
----------------------------------------
Ran 2 tests in 0.000s

OK
```

A unittest Test Case: setUp and Checks

tests/test_account.py

```python
import unittest

from banking import Account, InsufficientBalance


class AccountTest(unittest.TestCase):
    def setUp(self):
        self.account = Account("TR1", 1000)

    def test_deposit_increases_balance(self):
        self.account.deposit(1)
        self.assertEqual(self.account.balance, 1001)

    def test_withdraw_all_leaves_zero(self):
        self.account.withdraw(1000)
        self.assertEqual(self.account.balance, 0)
```

Key idea

`setUp` runs before **each** test method, so every test starts with a fresh account.

A unittest Test Case: Expected Exceptions

tests/test_account.py (continued)

```python
class AccountTest(unittest.TestCase):
    ...  # setUp and the tests from the previous slide

    def test_withdraw_rejects_negative_amount(self):
        with self.assertRaises(ValueError) as cm:
            self.account.withdraw(-1)
        self.assertEqual(str(cm.exception), "amount must be positive")
        self.assertEqual(self.account.balance, 1000)

    def test_withdraw_over_balance_raises(self):
        with self.assertRaises(InsufficientBalance) as cm:
            self.account.withdraw(1001)
        self.assertEqual(cm.exception.deficit, 1)
        self.assertEqual(self.account.balance, 1000)


if __name__ == "__main__":
    unittest.main()
```

Watch out

Only the call that should raise goes inside the `with` block; inspect `cm.exception` *after* the block.

unittest Assertion Methods

Method	Checks that
`assertEqual(a, b)`	`a == b`
`assertNotEqual(a, b)`	`a != b`
`assertTrue(x)`	`bool(x) is True`
`assertFalse(x)`	`bool(x) is False`
`assertIs(a, b)`	`a is b`
`assertIsNone(x)`	`x is None`

Method	Checks that
`assertIn(a, b)`	`a in b`
`assertIsInstance(a, cls)`	`isinstance(a, cls)`
`assertAlmostEqual(a, b)`	`round(a - b, 7) == 0`
`assertCountEqual(a, b)`	same elements, any order
`assertRaises(E)`	the block raises `E`
`assertRaisesRegex(E, r)`	…and the message matches `r`

Tip

Prefer the most specific assertion: `assertIn(x, items)` reports both values when it fails, `assertTrue(x in items)` only says `False is not true`.

Running unittest from the Command Line

Command	What it runs
`python -m unittest test_account`	one module (or `test_account.AccountTest.test_x`)
`python -m unittest -v test_account`	verbose: one line per test
`python -m unittest discover -s tests`	every `test*.py` file found below `tests/`
`python test_account.py`	`unittest.main()` inside the `__main__` guard

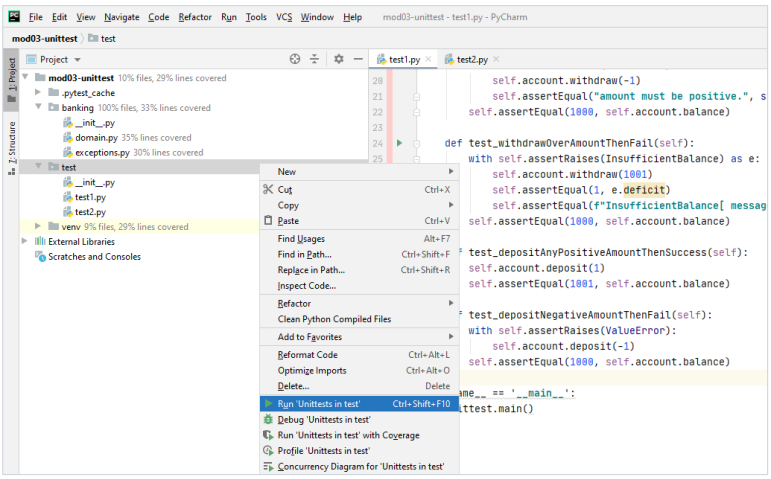
TERMINAL

```
$ python -m unittest discover -s tests -v
test_deposit_increases_balance (test_account.AccountTest.test_deposit_increases_balance) ... ok
test_withdraw_all_leaves_zero (test_account.AccountTest.test_withdraw_all_leaves_zero) ... ok
test_withdraw_over_balance_raises (test_account.AccountTest.test_withdraw_over_balance_raises) ... ok
test_withdraw_rejects_negative_amount (test_account.AccountTest.test_withdraw_rejects_negative_amount) ... ok

----------------------------------------------------------------------
Ran 4 tests in 0.001s

OK
```

Running Tests from PyCharm



Right-click the tests folder → Run 'Unittests in test'

- Green arrows in the gutter run a single test or class
- *Run with Coverage* shows covered lines in the editor
- Failed tests link straight to the failing line
- Choose the runner: *Settings → Tools → Python Integrated Tools*

FIND THE BUG

Find the Bugs in This unittest Test

```python
import unittest
import coverage


class AccountTest(unittest.TestCase):
    def setUp(self):
        self.account = Account("TR1", 1000)
        self.assertEqual(1000, self.account.balance)

    def test_withdrawNegativeAmountThenFail(self):
        with self.assertRaises(ValueError) as e:
            self.account.withdraw(-1)
            self.assertEqual("amount must be positive", str(e))
        self.assertEqual(1000, self.account.balance)

    def test_withdrawOverAmountThenFail(self):
        with self.assertRaises(InsufficientBalance) as e:
            self.account.withdraw(1001)
            self.assertEqual(1, e.deficit)
        self.assertEqual(1000, self.account.balance)
```

ANSWER

Find the Bugs in This unittest Test

- Lines 13 and 19 never run: the exception leaves the `with` block first
- `e` is the context object, not the exception: use `e.exception`
- `Account` and `InsufficientBalance` are never imported; `coverage` is unused
- `setUp` contains an assertion: test logic in the fixture
- camelCase names; prefer `test_withdraw_over_balance_raises`

fixed

```
def test_withdraw_over_balance_raises(self):
    with self.assertRaises(InsufficientBalance) as cm:
        self.account.withdraw(1001)
    self.assertEqual(cm.exception.deficit, 1)
    self.assertEqual(self.account.balance, 1000)
```


05

pytest Essentials

Plain functions, plain assert, powerful tooling

- Installing and configuring pytest
- Test discovery rules
- Assertion introspection
- Running and selecting tests

Why pytest?

- Tests are plain functions: no classes, no `self`
- Plain `assert` with detailed failure reports
- Fixtures: reusable, composable setup and teardown
- Parametrization, markers, rich test selection
- Over a thousand plugins: `pytest-cov`, `pytest-mock`, `pytest-xdist` (parallel runs)
- Runs existing `unittest` suites unchanged

TERMINAL

```
$ python -m venv .venv
$ source .venv/bin/activate
$ python -m pip install -U pytest
$ pytest --version
pytest 8.3.5
```

pyproject.toml

```toml
[tool.pytest.ini_options]
testpaths = ["tests"]
pythonpath = ["."]
addopts = "-ra"
```


Test Discovery Rules

project layout

```
bank-app/
├── pyproject.toml
├── banking.py
└── tests/
    ├── conftest.py
    ├── test_account.py
    └── transfer_test.py
```

Item	Collected when
File	`test_*.py` or `*_test.py`
Function	name starts with `test`
Class	name starts with `Test`, no `__init__`
Method	starts with `test`, inside a `Test*` class
Directory	searched recursively (skips `.venv`, `build`, ...)

Tip

Patterns are configurable: `python_files`, `python_classes`, `python_functions`.

Your First pytest Tests

tests/test_account.py

```python
from banking import Account


def test_deposit_increases_balance():
    account = Account("TR1", 1000)
    account.deposit(1)
    assert account.balance == 1001


def test_withdraw_all_leaves_zero():
    account = Account("TR1", 1000)
    account.withdraw(1000)
    assert account.balance == 0


def test_new_account_is_empty():
    assert Account("TR2").balance == 0
```

TERMINAL

```
$ pytest -q tests/test_account.py
...                                                  [100%]
3 passed in 0.02s
```

Key idea

No `import pytest`, no class, no `self`: a test is a function with an `assert`.

Note

pytest also runs the `unittest.TestCase` classes from section 4 unchanged.

Plain assert with Introspection

tests/test_intro.py

```python
from banking import Account


def test_deposit_increases_balance():
    account = Account("TR1", 1000)
    account.deposit(100)
    assert account == Account("TR1", 1110)
```

TERMINAL (ABRIDGED)

```
$ pytest -q tests/test_intro.py
F                                                    [100%]
>       assert account == Account("TR1", 1110)
E       AssertionError: assert Account(iban=...VE: 'active'>) == Account(iban=...VE: 'active'>)
E         Omitting 2 identical items, use -vv to show
E         Differing attributes:
E         ['balance']
E         Drill down into differing attribute balance:
E           balance: 1100 != 1110
1 failed in 0.08s
```

QUIZ

Which Tests Does pytest Collect?

Which functions in `tests/test_basic.py` does pytest collect?

Function in the file	Its body
`def test_sqrt():`	`assert 25 ** 0.5 == 5`
`def testsquare():`	`assert 7 * 7 == 40`
`def tesequality():`	`assert 10 == 11`

A. Only `test_sqrt`

B. `test_sqrt` and `testsquare`

C. All three functions

D. None: tests need a `Test` class

ANSWER

Which Tests Does pytest Collect?

Which functions in `tests/test_basic.py` does pytest collect?

Function in the file	Its body
`def test_sqrt():`	`assert 25 ** 0.5 == 5`
`def testsquare():`	`assert 7 * 7 == 40`
`def tesequality():`	`assert 10 == 11`

A. Only `test_sqrt`

B. `test_sqrt` and `testsquare`

C. All three functions

D. None: tests need a `Test` class

★ **Why**

Names *starting with* `test` are collected: `testsquare` runs and fails (49 ≠ 40); `tesequality` never runs.

Running and Selecting Tests

Command	Effect
`pytest`	all tests below `testpaths` (or the current directory)
`pytest tests/test_account.py`	one file
`pytest tests/test_account.py::test_deposit_increases_balance`	one test (its *node id*)
`pytest -k "withdraw and not negative"`	tests whose names match the expression
`pytest -m slow`	tests with a marker (section 6)
`pytest -x` · `--maxfail=2`	stop after the first · second failure
`pytest -q` · `-v`	quieter · verbose output
`pytest --lf`	re-run only the tests that failed last time

TRY IT

Write Your First pytest Tests

Write `tests/test_leap.py` with one test per rule, then run `pytest -v`.

leap.py

```python
def is_leap_year(year: int) -> bool:
    return year % 4 == 0 and (year % 100 != 0 or year % 400 == 0)
```

- Divisible by 4 → leap year
- Not divisible by 4 → common year
- Century (divisible by 100) → common year …
- … unless divisible by 400 → leap year
- Bonus: delete `or year % 400 == 0` — which test turns red?

06

Fixtures and Parametrization

Reusable setup, many cases, fine-grained control

- Fixtures, yield teardown and scope
- conftest.py and built-in fixtures
- Parametrized tests with readable ids
- Markers: skip, skipif, xfail

Fixtures: Reusable Arrange Code

tests/test_account.py

```python
import pytest

from banking import Account


@pytest.fixture
def an_active_account():
    return Account("TR1", 1000)


def test_withdraw_all_balance_leaves_zero(an_active_account):
    an_active_account.withdraw(1000)
    assert an_active_account.balance == 0


def test_deposit_increases_balance(an_active_account):
    an_active_account.deposit(1)
    assert an_active_account.balance == 1001
```

Key idea

pytest matches the **parameter name** to the fixture, calls it and passes the result — a fresh account for every test.

Fixtures: Teardown with yield

tests/test_db.py

```python
import sqlite3

import pytest


@pytest.fixture
def db():
    connection = sqlite3.connect(":memory:")
    connection.execute("CREATE TABLE accounts (iban TEXT, balance REAL)")
    yield connection          # the test runs here
    connection.close()        # teardown: runs even if the test fails


def test_insert_account(db):
    db.execute("INSERT INTO accounts VALUES ('TR1', 1000)")
    (count,) = db.execute("SELECT COUNT(*) FROM accounts").fetchone()
    assert count == 1
```

Key idea

Code before `yield` = setup, the yielded value goes to the test, code after `yield` = teardown.

Fixtures: Scope

Scope	Created once per	Typical use
`function`	test (default)	mutable objects: accounts
`class`	test class	data shared by a class
`module`	test file	expensive per-file setup
`package`	test package	package-wide test data
`session`	whole test run	server, container, database

tests/test_db.py

```python
@pytest.fixture(scope="module")
def db():
    connection = sqlite3.connect(":memory:")
    yield connection
    connection.close()
```

Watch out

Wider scope = shared state: tests must not change what the fixture holds.

Sharing Fixtures with `conftest.py`

project layout

```
bank-app/
├── banking.py
├── pyproject.toml
└── tests/
    ├── conftest.py      ← fixtures
    ├── test_account.py
    └── test_transfer.py
```

Key idea

No import needed: fixtures in `conftest.py` are available to every test in that directory and below.

tests/conftest.py

```python
import pytest

from banking import Account, AccountStatus


@pytest.fixture
def an_active_account():
    return Account("TR1", 1000)


@pytest.fixture
def a_closed_account():
    return Account("TR2", 1000, AccountStatus.CLOSED)
```


Built-in Fixtures: `tmp_path` and `capsys`

`tmp_path`: a new, empty `pathlib.Path` directory for each test

```python
def test_save_statement(tmp_path):
    target = tmp_path / "statement.txt"
    save_statement(target, ["TR1 1000", "TR2"])
    text = target.read_text(encoding="utf-8")
    assert text == "TR1 1000\nTR2"
```

`capsys`: captures what the code prints

```python
def test_print_balance(capsys):
    print_balance(Account("TR1", 1000))
    captured = capsys.readouterr()
    assert captured.out == "TR1: 1000.00 TL\n"
```

statements.py (code under test)

```python
def save_statement(path: Path, lines: list[str]) -> None:
    path.write_text("\n".join(lines), encoding="utf-8")


def print_balance(account: Account) -> None:
    print(f"{account.iban}: {account.balance:.2f} TL")
```

Built-in Fixtures: `monkeypatch`

tests/test_config.py

```python
import os


def api_url() -> str:
    return os.environ.get("BANK_API_URL", "https://api.example.com")


def test_api_url_from_environment(monkeypatch):
    monkeypatch.setenv("BANK_API_URL", "http://localhost:8000")
    assert api_url() == "http://localhost:8000"


def test_api_url_default(monkeypatch):
    monkeypatch.delenv("BANK_API_URL", raising=False)
    assert api_url() == "https://api.example.com"
```

Note

Also `setattr(obj, name, value)`, `setitem(mapping, key, value)`, `chdir(path)` and `syspath_prepend(path)` — every change is undone after the test.

Parametrized Tests

tests/test_param.py

```python
import pytest

DEPOSITS = [(0.1, 1000.1), (1, 1001), (100, 1100)]


@pytest.mark.parametrize(("amount", "expected"), DEPOSITS)
def test_deposit_increases_balance(an_active_account, amount, expected):
    an_active_account.deposit(amount)
    assert an_active_account.balance == pytest.approx(expected)
```

TERMINAL

```
$ pytest -v tests/test_param.py
tests/test_param.py::test_deposit_increases_balance[0.1-1000.1] PASSED   [ 33%]
tests/test_param.py::test_deposit_increases_balance[1-1001] PASSED       [ 66%]
tests/test_param.py::test_deposit_increases_balance[100-1100] PASSED     [100%]
```

Tip

`pytest.approx` compares floats with a tolerance: `0.1 + 0.2 == 0.3` is `False`.

Parametrize: ids and pytest.param

tests/test_param.py

```python
@pytest.mark.parametrize(
    ("amount", "expected"),
    [
        pytest.param(1, 999, id="one"),
        pytest.param(999, 1, id="almost-all"),
        pytest.param(1000, 0, id="everything"),
    ],
)
def test_withdraw_decreases_balance(an_active_account, amount, expected):
    an_active_account.withdraw(amount)
    assert an_active_account.balance == expected
```

TERMINAL

```
$ pytest -v tests/test_param.py -k withdraw
tests/test_param.py::test_withdraw_decreases_balance[one] PASSED         [ 33%]
tests/test_param.py::test_withdraw_decreases_balance[almost-all] PASSED  [ 66%]
tests/test_param.py::test_withdraw_decreases_balance[everything] PASSED  [100%]
```


Markers: skip, skipif and xfail

tests/test_markers.py

```python
import sys

import pytest


@pytest.mark.skip(reason="exchange service is down")
def test_live_exchange_rate():
    ...


@pytest.mark.skipif(sys.platform == "win32",
                    reason="POSIX only")
def test_posix_file_permissions():
    ...


@pytest.mark.xfail(reason="2.675 is stored as 2.67499...")
def test_round_half_cent():
    assert round(2.675, 2) == 2.68
```

Letter	Outcome
`s`	skipped: not run
`x`	xfail: failed as expected
`X`	xpass: passed unexpectedly

custom marker

```python
@pytest.mark.slow
def test_monthly_interest_for_all():
    ...
```

pyproject.toml

```toml
[tool.pytest.ini_options]
markers = ["slow: takes more than a second"]
```

TERMINAL

```
$ pytest -m "not slow" -rsx
```

TRACE THE CODE

In Which Order Are the Lines Printed?

tests/test_order.py

```python
import pytest


@pytest.fixture(scope="module")
def bank():
    print("open bank")
    yield
    print("close bank")


@pytest.fixture
def account(bank):
    print("new account")
    yield
    print("drop account")
```

tests/test_order.py (continued)

```python
def test_deposit(account):
    print("test_deposit")


def test_withdraw(account):
    print("test_withdraw")
```

TERMINAL

```
$ pytest -s -q tests/test_order.py
```

Note

`-s` turns off output capturing, so `print` output appears in the terminal.

ANSWER

In Which Order Are the Lines Printed?

OUTPUT

```
open bank
new account
test_deposit
.drop account
new account
test_withdraw
.drop account
close bank

2 passed in 0.01s
```

- `bank` (module scope) is set up **once**, before the first test
- `account` (function scope) is set up and torn down around **each** test
- Teardown runs in reverse order: `account` first, `bank` last
- The `.` is pytest's progress mark for each passed test

07

Testing Exceptions and Mocking

Expected failures and isolated dependencies

- pytest.raises with match
- unittest.mock: Mock, patch and assertions on calls
- Where to patch
- pytest-mock and the mocker fixture

Testing Exceptions with `pytest.raises`

tests/test_raises.py

```python
import pytest

from banking import InsufficientBalance


def test_withdraw_negative_amount_raises(an_active_account):
    with pytest.raises(ValueError, match="must be positive"):
        an_active_account.withdraw(-1)


def test_withdraw_over_balance_reports_deficit(an_active_account):
    with pytest.raises(InsufficientBalance) as excinfo:
        an_active_account.withdraw(1001)
    assert excinfo.value.deficit == 1
    assert an_active_account.balance == 1000
```

Note

`with` uses `pytest.raises` as a *context manager* (formally covered in Module 4). If nothing is raised the test fails with `DID NOT RAISE`.

unittest.mock: Programmable Test Doubles

```
>>> from unittest.mock import Mock
>>> rates = Mock()
>>> rates.fetch_rate.return_value = 35.5
>>> rates.fetch_rate("USD", "TRY")
35.5
>>> rates.fetch_rate.assert_called_once_with("USD", "TRY")
>>> rates.fetch_rate.call_count
1
>>> rates.fetch_rate.side_effect = ConnectionError("offline")
>>> rates.fetch_rate("EUR", "TRY")
Traceback (most recent call last):
  ...
ConnectionError: offline
```

Attribute / method	Purpose
`return_value`	canned answer (stub)
`side_effect`	raise, or return values in turn
`assert_called_once_with(...)`	verify the call (mock)
`assert_not_called()`	verify no call happened
`call_args`, `call_count`	inspect calls (spy)
`Mock(spec=Cls)`	only allow attributes of `Cls`

Patching Dependencies with `patch`

rates.py

```python
import json
from urllib.request import urlopen

API = "https://api.example.com/rates"


def fetch_rate(source: str, target: str) -> float:
    with urlopen(f"{API}/{source}/{target}") as resp:
        return json.load(resp)["rate"]
```

converter.py

```python
from rates import fetch_rate


def convert(amount: float, source: str,
            target: str) -> float:
    return round(amount * fetch_rate(source, target), 2)
```

tests/test_converter.py

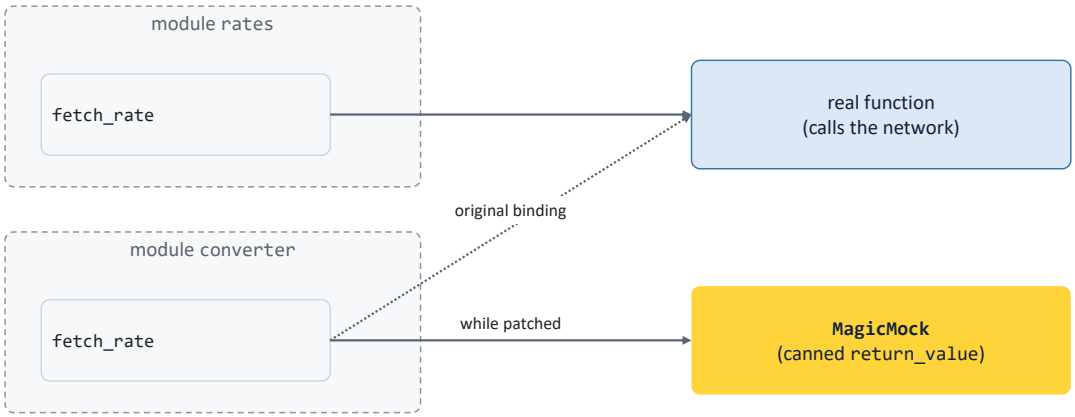
```python
from unittest.mock import patch

from converter import convert


def test_convert_uses_current_rate():
    with patch("converter.fetch_rate") as fake:
        fake.return_value = 35.5
        assert convert(10, "USD", "TRY") == 355.0
    fake.assert_called_once_with("USD", "TRY")


@patch("converter.fetch_rate", return_value=1.23456)
def test_convert_rounds_to_cents(fake):
    assert convert(1, "EUR", "USD") == 1.23
```

Patch Where the Name Is Looked Up



Key idea

`from rates import fetch_rate` copies the *name* into `converter`. Patch `"converter.fetch_rate"` — the name `convert` actually looks up.

pytest-mock: The `mocker` Fixture

tests/test_mocker.py

```python
from banking import Account
from converter import convert
from transfers import transfer


def test_convert_with_mocker(mocker):
    fake = mocker.patch("converter.fetch_rate", return_value=35.5)
    assert convert(10, "USD", "TRY") == 355.0
    fake.assert_called_once_with("USD", "TRY")


def test_transfer_withdraws_from_source(mocker):
    source, target = Account("TR1", 1000), Account("TR2")
    spy = mocker.spy(source, "withdraw")
    transfer(source, target, 250)  # withdraw, then deposit
    spy.assert_called_once_with(250)
    assert (source.balance, target.balance) == (750, 250)
```

Tip

`pip install pytest-mock` — no `with`, no decorator: every patch is undone automatically after the test.

QUIZ

Where Should the Test Patch?

`report.py` does `from rates import fetch_rate` and calls `fetch_rate()` inside `daily_report()`. Which target should the test patch?

A `"rates.fetch_rate"`

B `"report.fetch_rate"`

C `"urllib.request.urlopen"`

D `"fetch_rate"`

ANSWER

Where Should the Test Patch?

`report.py` does `from rates import fetch_rate` and calls `fetch_rate()` inside `daily_report()`. Which target should the test patch?

A `"rates.fetch_rate"`

B `"report.fetch_rate"`

C `"urllib.request.urlopen"`

D `"fetch_rate"`

★ **Why**

Patch where the name is looked up: `daily_report` finds `fetch_rate` in the namespace of `report`. Patching `rates.fetch_rate` leaves that copy untouched; a bare name is not a valid target.

TRY IT

Verify an Alert with a `Mock`

alerts.py

```
def alert_if_low(account, notifier, limit=100):
    if account.balance < limit:
        notifier.send(account.iban, "Low balance")
```

1. Test 1: balance 40 → `send` called once with `("TR1", "Low balance")`
2. Test 2: balance 900 → `send` never called
3. Use `unittest.mock.Mock` for the notifier and real `Account` objects

Hint

No `patch` needed: the notifier is a parameter, so simply pass a `Mock()` in its place.

ANSWER

Verify an Alert with a `Mock`

tests/test_alerts.py

```
from unittest.mock import Mock

from alerts import alert_if_low
from banking import Account


def test_low_balance_sends_alert():
    notifier = Mock()
    alert_if_low(Account("TR1", 40), notifier)
    notifier.send.assert_called_once_with("TR1", "Low balance")


def test_healthy_balance_sends_nothing():
    notifier = Mock()
    alert_if_low(Account("TR2", 900), notifier)
    notifier.send.assert_not_called()
```

08

Coverage and TDD

Measure what the tests exercise, let tests drive the design

- Line and branch coverage with pytest-cov
- What coverage does not tell you
- Red–green–refactor
- A worked TDD example

Measuring Coverage with pytest-cov

Suite: only the happy-path tests from sections 5–6

TERMINAL (ABRIDGED)

```
$ python -m pip install pytest-cov
$ pytest -q --cov=banking --cov-report=term-missing
...........                                                  [100%]
Name         Stmts   Miss  Cover   Missing
----------------------------------------
banking.py      27      6    78%   14-16, 32, 37, 39
----------------------------------------
TOTAL           27      6    78%
11 passed in 0.21s
```

Key idea

The **Missing** column is a to-do list: lines 14–16 and 32 are the over-balance path, 37 the closed account, 39 the non-positive amount.

Coverage Is Not Quality

fee.py

```python
def fee(amount: float, premium: bool) -> float:
    rate = 0.01
    if premium:
        rate = 0.0
    return amount * rate
```

tests/test_fee.py

```python
from fee import fee


def test_premium_fee():
    fee(100, premium=True)
```

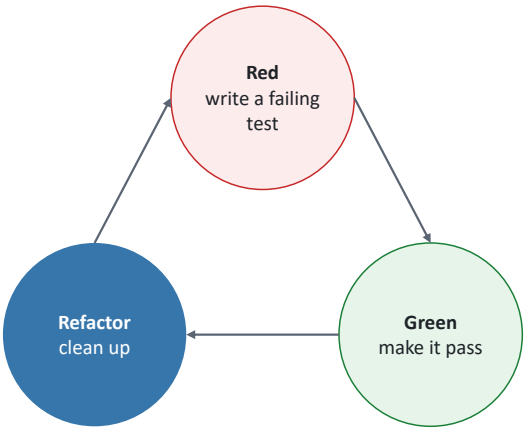
TERMINAL (ABRIDGED)

```
$ pytest -q --cov=fee --cov-branch \
    --cov-report=term-missing
.                                   [100%]
Name     Stmts   Miss Branch BrPart  Cover   Missing
-------------------------------------------------------
fee.py       5      0      2      1    86%   3->5
-------------------------------------------------------
TOTAL        5      0      2      1    86%
1 passed in 0.05s
```

✗ **Common misconception**

Every line ran, yet the test checks nothing — it has no `assert`. Coverage shows what *ran*, not what was *verified*.

Test-Driven Development: Red, Green, Refactor



- **Red**: write a small test for the next behaviour; watch it fail
- **Green**: write the simplest code that makes it pass
- **Refactor**: improve code *and* tests while everything stays green
- Repeat in cycles of minutes, not hours
- Tests become an executable specification (Beck, 2002)

TDD Cycle 1: Red, Then Green

Red: the test comes first

tests/test_password.py

```python
from password import is_strong_password


def test_short_password_is_weak():
    assert not is_strong_password("Ab1")
```

TERMINAL (ABRIDGED)

```
$ pytest -q
E   ModuleNotFoundError: No module named 'password'
1 error in 0.14s
```

Green: the simplest code that passes

password.py

```python
def is_strong_password(password: str) -> bool:
    return len(password) >= 8
```

TERMINAL

```
$ pytest -q
.                                                    [100%]
1 passed in 0.01s
```


TDD Cycle 2: A New Failing Test

Red: add a test for the digit rule

tests/test_password.py (added)

```python
def test_password_without_digit_is_weak():
    assert not is_strong_password("Abcdefgh")
```

TERMINAL (ABRIDGED)

```
$ pytest -q
.F                                                   [100%]
E       AssertionError: assert not True
E        +  where True = is_strong_password('Abcdefgh')
1 failed, 1 passed in 0.05s
```

Green: extend the implementation

password.py

```python
def is_strong_password(password: str) -> bool:
    has_digit = any(ch.isdigit() for ch in password)
    return len(password) >= 8 and has_digit
```

TERMINAL

```
$ pytest -q
..                                                   [100%]
2 passed in 0.01s
```

TDD Cycle 3: Refactor Safely

password.py

```python
MIN_LENGTH = 8


def has_min_length(password: str) -> bool:
    return len(password) >= MIN_LENGTH


def has_digit(password: str) -> bool:
    return any(ch.isdigit() for ch in password)


RULES = [has_min_length, has_digit]


def is_strong_password(password: str) -> bool:
    """Return True if every rule accepts the password."""
    return all(rule(password) for rule in RULES)
```

tests/test_password.py

```python
import pytest

from password import is_strong_password


@pytest.mark.parametrize(
    "password",
    ["Ab1", "Abcdefgh", ""],
    ids=["too-short", "no-digit", "empty"],
)
def test_weak_password_is_rejected(password):
    assert not is_strong_password(password)


def test_long_password_with_digit_is_strong():
    assert is_strong_password("Abcdefg1")
```

TERMINAL

```
$ pytest -q
....                                [100%]
4 passed in 0.02s
```


QUIZ

What Does 100 % Coverage Mean?

A test suite reaches 100 % line coverage. What can you conclude?

A. The code has no bugs

B. Every line ran at least once during the tests

C. Every `if` was evaluated both ways

D. Every requirement is tested

ANSWER

What Does 100 % Coverage Mean?

A test suite reaches 100 % line coverage. What can you conclude?

- A The code has no bugs
- **B Every line ran at least once during the tests**
- C Every `if` was evaluated both ways
- D Every requirement is tested

Why

Line coverage only records execution. Branches may be taken one way only, assertions may be missing, and requirements that were never implemented have no code to cover.

8 · COVERAGE AND TDD

Practice Problems

1 · TDD

Add the rule *contains an uppercase letter* to `is_strong_password`, test first

2 · Fixtures

Write tests for `transfer` using the `conftest.py` accounts, incl. a closed target account

3 · Parametrize

Turn the four leap-year tests into one parametrized test with ids

4 · Mocking

Test that `convert` propagates `ConnectionError` using `side_effect`

5 · Coverage

Bring `banking.py` to 100 % *branch* coverage; list the tests you needed

6 · Static analysis

Run `ruff check` and `mypy` on your project and fix every finding

Tip

Work in a virtual environment with `pytest`, `pytest-cov` and `pytest-mock` installed, and run the whole suite after every change.

MODULE 03 · SUMMARY

Key Takeaways

- ISO/IEC 25010 defines quality; verification checks the spec, validation the real need
- Errors inject faults that cause failures; static analysis (`ruff`, `mypy`) and tests cut the chain
- Many fast unit tests at the base of the pyramid; doubles isolate the unit under test
- Good tests: Arrange–Act–Assert, one behaviour each, fast, independent, repeatable
- pytest: plain `assert`, fixtures with `yield` and scope, `parametrize`, markers
- `pytest.raises` for expected exceptions; patch where the name is looked up
- Coverage finds untested code but does not prove correctness; TDD grows tested code step by step

NEXT UP Module 04 — File Operations in Python